Tarea 2 Komikan

Nombre: Ignacio Espinoza
RUT: 22.024.681-7
Sección: 3

Universidad Diego Portales
Sistemas Operativos
Otoño 2026

Introducción

Juego concurrente de Komikan echo con hilos. Este simula partidas en tiempo real entre 13 fichas, 12 perros y 1 puma, cada ficha se mueve independientemente segun una funcion random. El objetivo fue lograr desarrollar un sistema que lograra correr en hilos varias mesas con sus jugadores y fichas que tambien son hilos asegurando una exclusion mutua y con objetivo de lograr sincronizar bien los movimientos de las fichas.

1. Protección del tablero

Cada tablero tiene un arreglo mutex, o sea un mutex por casilla. No hay mutex global por tablero, logrando que fichas en diferentes partes del tablero puedan moverse en tiempo real.

2. Diseño del Esquema de Sincronización

2.1. Movimientos

Se desarrolló una funcion que copia los IDs de las casillas involucradas con un movimiento a un arreglo local, los ordena con sort para garantizar que los mutex siempre se tomen en el mismo orden global, evitando así deadlocks entre hilos que compiten por casillas en común usando trylock. Si falla una casilla, libera las tomadas y retorna false. Si las toma todas, verifica si terminó la partida, si terminó libera todo y retorna false, en caso contrario se mueve.

2.2. Juez y fichas

Tras bloquear las casillas, las fichas delegan la validación y ejecución de sus movimientos al juez bajo un mutex global. Este proceso centralizado aplica las reglas, actualiza el estado y evalúa las condiciones de término (capturas, acorralamiento o límite de movimientos). Si la partida finaliza, se descartan jugadas posteriores. Por su parte, el hilo del juez, sincronizado mediante una variable de condición, se encarga de registrar el historial y emitir el resultado, incluyendo las victorias por desconexión.

2.3. Mesas

Se utiliza un semáforo para limitar la cantidad de mesas simultáneas a un máximo de M. El registro y la eliminación de las mesas activas en el sistema se protegen mediante un mutex.

Para manejar la señal de interrupción Ctrl+C, un hilo dedicado espera el evento. Al recibirlo, bloquea la creación de nuevas mesas, marca las activas como terminadas y emite un broadcast para despertar a los jueces, garantizando así un cierre ordenado.

3. Análisis de Riesgos de Concurrencia

3.1. Deadlock

Se evitan deadlocks bloqueando las casillas en orden ascendente mediante trylock con rollback. Además, se respeta una estricta jerarquía de mutex, casillas antes de mutexAviso, luego mutexConsola y mutexTorneo previo a mutexAviso,, imposibilitando cualquier espera circular.

3.2. Livelock

Para prevenir livelocks al adquirir las casillas, se limitan los reintentos. Si estos fallan, el hilo se suspende por un tiempo aleatorio, lo que desfasa la ejecución, rompe la simetría y evita bucles de espera activa.

3.3. Starvation

Dado que trylock y los mutex POSIX no garantizan *fairness* ni colas FIFO, existe riesgo de inanición temporal. Se implementaron retardos aleatorios y un límite de intentos para mitigarlo, reconociéndose su persistencia como una limitación inherente al modelo no determinista.

3.4. Resiliencia

Para Ctrl+C, hiloSigint activa apagando, meterMesa rechaza mesas nuevas y las activas reciben terminada = true con broadcast. Para abandono existe avisarCaida, que toma mutexAviso, registra mesa.abandonaron y hace pthread_cond_signal(&mesa.avisoAlJuez), el Juez lee abandonaron y declara ganador al oponente. Sin embargo, la caída está simulada con mesaQueSeCae y fichaQueSeCae en -1, no hay detector real de desconexión. El mecanismo existe, pero no está cableado a una falla real.

4. Evaluación

4.1. Decisiones de diseño

Se optó por utilizar un mutex por casilla en lugar de uno global, sacrificando simplicidad para maximizar la concurrencia en la competencia por las casillas. La validación, aplicación y registro de cada movimiento se realiza de forma atómica bajo mutexAviso, lo que mantiene el estado consistente y el historial en el orden real, a costa de serializar esa sección crítica corta dentro de cada mesa.

4.2. Rendimiento

El rendimiento es óptimo al operar en zonas distintas. La eficiencia disminuye por las disputas de casillas y la serialización al confirmar movimientos bajo el mutex global. Aunque esta sección crítica es breve, carece de mediciones formales de *throughput*. Finalmente, un semáforo limita las mesas simultáneas para evitar saturar el sistema.

4.3. Limitaciones

No se modelaron saltos diagonales a los tronos desde (0,3) y (4,3). La detección de desconexiones opera con valores estáticos simulados. Finalmente, el juez actúa como función invocada por las fichas en su sección crítica, mientras que su hilo dedicado solo escribe el historial y reporta el resultado.